Troubleshooting of Imputation Baseline

- Issue
- Differences Between Implementations
- Experiments and Results with Author Repository
- Experiments and Results with Our Repository
- Conclusions
- Path Forward

Issue

- Initial naive implementation of WAPE/WPE for our internal DLinear baseline gave very different values from the reported numbers in the dataset authors' paper

- Troubleshooting of this difference is crucial for the validity of the comparisons to prior work

Main Differences

- Author environment is slightly different and older
  - Python 3.8 vs 3.12.3
  - Corresponding package environment is older as well
  - Older PyPOTS imputation library not installed, but exists as a subdirectory of the repo
- Author code uses 4 out of the 7 covariates available
  - 'discount', 'holiday_flag', 'precpt', 'avg_temperature'
- Author code pools residuals across days before taking absolute values
  - WAPE errors in different directions partially cancel, WPE unchanged
- Author code trains and tests on the same data

Author Repository Experiments

- Built WAPE/WPE function that can operate both with our PyPOTS output and their PyPOTS output
- Toggle
  - 4 vs. 7 covariates
  - Daily pooling vs. absolute hourly total residual for WAPE
    - New function only
  - Conda vs. uv environment manager
- 7 random seeds

Author Original Metric Function

Build Tool	Number Covariates	WAPE Mean (%)	WAPE Standard Deviation (%)	WPE Mean (%)	WPE Standard Deviation (%)	Sample Size
Conda	4	31.3115	1.6131	-19.7314	4.3137	7
Conda	7	31.7340	2.1640	-21.3321	4.8301	7
uv	4	31.4299	1.5594	-19.5986	4.3550	7
uv	7	31.7016	2.1664	-21.1980	4.8098	7

Author Reported WAPE: 29.99%
Author Reported WPE: -1.28%

All Metric Functions

Function Version	Pooled Before Absolute Value Calculation	WAPE Mean (%)	WAPE Standard Deviation (%)	WPE Mean (%)	WPE Standard Deviation (%)	Sample Size
Old	True	31.5443	1.7987	-20.4650	4.3981	28
New	True	33.2621	1.8964	-21.5795	4.6374	28
New	False	71.3069	1.9773	-21.5795	4.6374	28

Author Reported WAPE: 29.99%
Author Reported WPE: -1.28%

Takeaways

- Build tool version doesn’t make a difference
- Number of covariates makes a slight difference
- Pooling hours across the day before taking absolute values for WAPE calculation approximately halves the estimated error
- New function has a practically significant difference of about 2%, but should be appropriate for our use

Experiments with Our Repository

- Still need to rule out differences in environment
  - Build tools shouldn't have an impact, but Python, PyTorch, and PyPOTS versions may
- Still need to assess impact of training and testing on same data
  - Not possible with the author repository

- Evaluated DLinear with almost the same hyperparameters across 9 random seeds and 6 splits
  - They pare the dataset down to open hours prior to feeding in data and have a window of 30 days vs. 28, leading to n_features = 480 vs. n_features = 672
  - SAITS loss is implemented slightly differently, but still uses MAE

Results

Training set	Pooled Before Absolute Value Calculation	WAPE Mean (%)	WAPE Standard Deviation (%)	WPE Mean (%)	WPE Standard Deviation (%)	Sample Size
Train	True	45.6891	6.9736	-34.0153	12.5639	54
Train	False	80.4161	3.3805	-34.0153	12.5639	54
Validation	True	42.0728	8.0549	-26.0103	17.9450	54
Validation	False	80.6901	5.1260	-26.0103	17.9450	54

Conclusions

- Differences in build tools, covariates used, and training set had no practical impact on WAPE or WPE
- Pooling errors across the days considered to get net error approximately halves WAPE with no impact on WPE
  - Days considered are those with masking but without stockout
  - Isolates imputation performance from reconstruction performance
- Significant difference remains between our implementation and their implementation
  - Package and Python versions, as well as difference in n_features, are the two main explanations remaining

Outcomes

- Time spent on this troubleshooting has significantly impacted work timelines
  - Further troubleshooting will jeopardize project completion
- Function development for imputation necessary because there is no built-in ground truth function in PyPOTS
  - Ground truth metric functions exist for PyTorch-Forecasting
- We were able to create a function to evaluate WAPE and WPE that works both in our repository and the author repository

Path Forward

- Direct comparison to author repository is not useful at this point
  - Our non-quantile DLinear imputation model will serve as baseline for imputation
  - Our non-quantile DLinear forecasting model using DLinear imputation model output will serve as baseline for forecasting
- We will use the developed function for imputation performance evaluation
  - Client decision needed to decide if to use pooled or non-pooled variant
- We need to implement MASE for imputation models
- We need to implement WPE for forecasting models
- We will use PyTorch-Forecasting functions for WAPE and MASE for forecasting models to reduce development work

Print Output For Author Repository

```
2026-11-08 12:08:44 [INFO]: DLinear initialized with the given hyperparameters, the number of trainable parameters: 465,362
_DLinear(
  (series_decomp): SeriesDecompositionBlock(
    (moving_avg): MovingAvgBlock(
      (avg): AvgPool1d(kernel_size=(17,), stride=(1,), padding=(0,))
    )
  )
  (backbone): BackboneDLinear(
    (linear_seasonal): Linear(in_features=480, out_features=480, bias=True)
    (linear_trend): Linear(in_features=480, out_features=480, bias=True)
  )
  (linear_seasonal_embedding): Linear(in_features=18, out_features=64, bias=True)
  (linear_trend_embedding): Linear(in_features=18, out_features=64, bias=True)
  (linear_seasonal_output): Linear(in_features=64, out_features=9, bias=True)
  (linear_trend_output): Linear(in_features=64, out_features=9, bias=True)
  (saits_loss_func): SaitsLoss()
)
```

Print Output For Our Repository

```
2026-11-08 12:02:41 [INFO]: DLinear initialized with the given hyperparameters, the number of trainable parameters: 908,114
_DLinear(
  (series_decomp): SeriesDecompositionBlock(
    (moving_avg): MovingAvgBlock(
      (avg): AvgPool1d(kernel_size=(17,), stride=(1,), padding=(0,))
    )
  )
  (backbone): BackboneDLinear(
    (linear_seasonal): Linear(in_features=672, out_features=672, bias=True)
    (linear_trend): Linear(in_features=672, out_features=672, bias=True)
  )
  (seasonal_saits_embedding): SaitsEmbedding(
    (embedding_layer): Linear(in_features=18, out_features=64, bias=True)
  )
  (trend_saits_embedding): SaitsEmbedding(
    (embedding_layer): Linear(in_features=18, out_features=64, bias=True)
  )
  (linear_seasonal_output): Linear(in_features=64, out_features=9, bias=True)
  (linear_trend_output): Linear(in_features=64, out_features=9, bias=True)
  (training_loss): SaitsLoss(
    (loss_calc_func): MAE()
  )
  (validation_metric): MSE()
)
```